Solution Architecture

Data Normalization and LLM Extraction Pipeline

> **Purpose**
>
> Convert each retailer product JSON record into two normalized CSV rows: one evaluated as a New Line and one as an Upgrade. Gemini interprets unstructured offers; Python validates, applies source-backed rules, and writes the fixed assignment schema.

End-to-end flow

```
Input JSON -> input checks -> compact offer payload -> Gemini structured JSON
     -> Pydantic response validation -> deterministic source rules
     -> currency validation and normalization -> two CSV rows per product
```

Architecture principles

- Hybrid responsibilities: use the model for language interpretation and Python for repeatable formatting, calculations, guards, and file output.
- Schema-first response: the model is asked for typed New Line and Upgrade objects, not free-form prose or CSV.
- Source grounding: selected assignment-critical values are checked against offer text and product variant after the model responds.
- Traceability: logs identify the configured model and product/variant, plus retries and output completion. Credentials and full offer payloads are not logged.

Concurrency and output

The application permits up to five concurrent Gemini requests and waits for all product extractions before writing the CSV. Each input product yields exactly two rows and all output rows share the ordered 29-column header list.

Code Responsibilities

How the Python modules work together

main.py

Command-line orchestration. Loads and checks the product array, reads the API key from the environment, creates the Gemini client with a request timeout, launches extraction tasks, normalizes two rows per product, and writes CSV. Reports a helpful message when the destination cannot be written.

config.py

Loads environment configuration, defines the model name, concurrency/retry limits, and the exact CSV column order.

prompts.py

Builds a compact model payload. Includes product name, brand, variant, flattened promotion text, and selected accessory/add-on fields. Defines extraction instructions and sends retry correction context.

schemas.py

Defines Pydantic models for each line type and the top-level response. Extra response fields are forbidden; reasoning is required but discarded from CSV.

extractor.py

Calls Gemini with JSON response MIME type and the adapted response schema. Parses JSON, applies source-grounded rules, validates monetary values, and retries failures with backoff/correction context.

source_rules.py

Uses conservative text patterns to override facts that can be read explicitly from offers: storage-specific discounts, the example service credit, a unique switcher amount, a trade-in maximum, and Points Plus totals.

normalizer.py

Validates positive monetary values, removes insignificant trailing zeroes without rounding, maps product/extraction values into named columns, and guarantees blank switcher fields on Upgrade rows.

test_normalizer.py

Regression tests for assignment example values, variant selection, plan-type mapping, currency formatting, exact headers, two-row output, logging, and actionable output permission errors.

Extraction Methodology

From noisy retailer text to typed fields

1. Prepare source facts

Python reads a product and builds a reduced payload. Offer titles, descriptions, raw content, and available modal content are combined so the model can evaluate eligibility and category. Product price, stock, shipping, and plan type remain deterministic Python inputs.

2. Ask Gemini for two evaluations

The system prompt asks Gemini to match the exact storage variant, separate offer categories, avoid unsupported amounts, and evaluate New Line and Upgrade independently. Temperature is set to zero; the API is asked for JSON conforming to the response schema.

3. Validate response shape

Pydantic requires the defined fields and rejects extras or incompatible values. This validates structure, not truth: a syntactically valid but incorrect amount can still pass this stage.

4. Apply source-backed corrections

Python scans offer text for narrowly defined patterns. When a unique explicit amount is found, the response field is corrected from the source. Ambiguous values are not guessed by these rules.

5. Validate and normalize

Monetary fields must parse as positive currency. Output formatting removes redundant zeros (for example, $200.00 becomes $200) but keeps significant decimals ($244.36). Missing values become blank cells.

6. Write the fixed CSV

Python creates New Line and Upgrade rows for each product and writes the exact 29 headers in the assignment order.

> **Error handling**
>
> Gemini calls use a configured timeout. Extraction failures and invalid structured values are retried; logs report product, variant, model, and retry status. The CSV is written only after extraction succeeds, avoiding a success-shaped partial output.

Assignment Example and Verification

Expected behavior for example.json (iPhone 17 Pro Max, 256 GB)

Source-grounded expected values

FIELD	NEW LINE	UPGRADE
Unique device discount	$660.36	blank
Service-line credit	$200	blank
Switcher	$800	blank
Maximum trade-in value	blank	blank
Points Plus finance total	$450	$250

The device offer lists separate credit totals for 256 GB, 512 GB, 1 TB, and 2 TB. The selected amount must match the product variant. The additional $5.56 monthly credit is described as $200 total and is routed to the service-line field, not added to the device discount. The switcher requires switching/port-in, so Upgrade is blank. The generic trade-in description gives no maximum, so Max TI is blank.

Verification status

- Automated tests assert assignment example values, variant-specific behavior, currency normalization, and line-type exclusions.
- CSV tests assert the exact 29 headers and two rows per product.
- Compilation and the focused test suite are expected to run in the project virtual environment.

> **Important limitation**
>
> This design improves deterministic compliance but cannot guarantee a perfect evaluator score. Regex rules intentionally cover explicit patterns; other offer categories and descriptions still depend on Gemini and should be checked against any new evaluation examples.

Run locally

```
python -m pip install -r requirements.txt
python main.py Input.json output.csv
python -m pytest -q test_normalizer.py
```